



Holdfast Security Assessment Report

June 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Overview
5. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
6. Findings Summary
7. Weaknesses
 - Transaction data handling does not match privacy policy disclosures
 - WebSocket upgrades rely on a spoofable Origin header for access control
 - Privacy mode is not consistently applied to transaction history amounts
 - Withdraw amount prefill can generate a value rejected by its parser
 - Split preflight integrity check does not validate the account-creation prelude

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report presents the results of the security assessment of Holdfast, a project developed by Valigator. Holdfast is a non-custodial Solana stake manager delivered as a Chrome extension, built for hardware wallet users, which exposes the full range of Solana stake operations including activation, splitting, merging, deactivation, withdrawal, and transfer of stake accounts.

During the assessment, we identified one low-severity issue along with four informational findings, primarily related to implementation details and security best practices. No critical, high, or medium-severity vulnerabilities were identified.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our assessment.

3. Security Review Details

- Review Led by

Jahyun Koo, Lead Security Researcher

- Scope

The analyzed resources are located on:

<https://github.com/valigator-tech/tooth>
Commit Hash: b983de14ab7c45f3c2bf513ceb01109982f28db6

<https://github.com/valigator-tech/rpc-proxy>
Commit Hash: fa117be0de95beb5e9e8c5c81147dc81be1afe02

The issues described in this report were fixed in the following commits:

<https://github.com/valigator-tech/tooth>
Commit Hash: ad75a27c19641022634d9a2816645be56872c85a

<https://github.com/valigator-tech/rpc-proxy>
Commit Hash: 0fbd9e31beef1df82d8e5fe842252b24341bcd5f

- Changelog

■ 22 June 2026	Audit start
■ 2 July 2026	Initial report
■ 6 July 2026	Revision received
■ 7 July 2026	Final report

4. Overview

▪ Access Control

We reviewed access control on both the wallet side and the RPC proxy. In the extension, signing flows are only reachable from first-party extension UI paths.

The extension exposes no injected wallet provider, no arbitrary dApp connection, and no content scripts, so external sites cannot directly request signatures.

Signing requests are passed through extension-controlled session state and are checked before hardware signing.

On the proxy side, HTTP requests are authenticated with the app token and HMAC signature, with Origin used as an additional check when present. The WebSocket path is weaker: it skips token and HMAC validation and relies only on Origin, which non-browser clients can spoof. This leaves a narrow subscribe-only path to the upstream WebSocket, with no fund-moving capability reachable through it (HOLD1-2).

▪ Phishing Protection

We reviewed dApp exposure, injected-provider behavior, and delegation destination handling. The extension exposes no injected wallet provider, no arbitrary dApp connection, and no content scripts. Delegation destinations are pinned to the Valigator vote account and checked by public key at signing time rather than by display name. We did not identify a reportable phishing exposure.

▪ Key Management & Cryptography

We reviewed key custody, hardware signing, signature verification, and transaction preflight checks. The extension does not create or store user seed material or private keys. Transactions are signed by an external hardware device, and returned signatures are verified against the exact message and public key before broadcast. The preflight checks bind the reviewed operation's destination, authority, amount, and validator fields. One completeness gap remains: the split flow does not validate its account-creation prelude the way create-delegate does (HOLD1-5).

- **Web Vulnerabilities**

We reviewed DOM sinks, dynamic code usage, output encoding, and CSP. Production code uses React rendering and does not use `dangerouslySetInnerHTML`, `direct innerHTML`, `document.write`, `eval`, or `new Function`. The extension CSP restricts scripts to the extension itself and blocks plugin content. We did not identify a web-layer injection issue.

- **Policy Consistency**

We compared the Privacy Policy and Chrome Web Store disclosures against RPC proxy behavior. The policy says wallet addresses and transaction data are not logged, stored, or transmitted, but transaction submissions are written to a wallet-indexed table with client IP, wallet address, and raw signed transaction body, and public keys/signatures are also sent to an operational webhook (HOLD1-1). The policy wording should be reconciled with the implementation.

- **UI and Display**

We reviewed privacy-mode coverage, input handling, and on-screen value accuracy. Privacy mode is not consistently applied to Dashboard Recent Activity or expanded Activity Log rows, so transaction amounts can remain visible (HOLD1-3). The withdraw form can also prefill a comma-formatted amount that its own parser rejects for large balances (HOLD1-4). Neither issue changes signed transaction contents or redirects funds.

- **Supply Chain and CI/CD Pipeline**

We reviewed dependency, build, and release controls, including publish-age cooldowns, lockfile and exact-version checks, Semgrep, CodeQL, OSV scanning, AI-assisted PR review, and third-party action pinning. Coverage is broad. The remaining weaknesses are process-hardening items reachable only by an internal or compromised PR author, with no identified shipped-product exposure.

5. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

6. Findings Summary

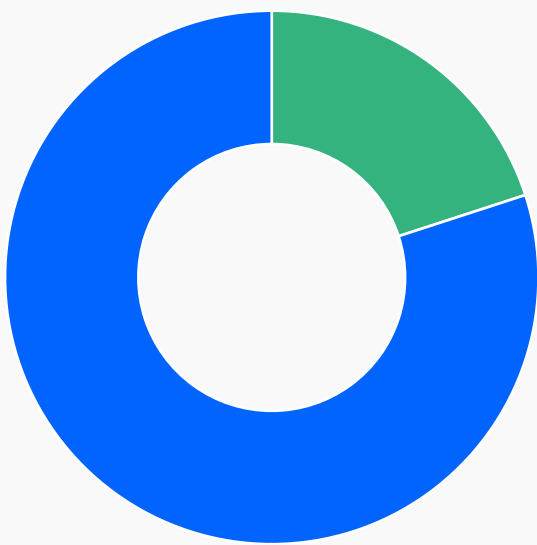
Severity

Number of findings

Critical	0
High	0
Medium	0
Low	1
Informational	4

Total:

5



Low

Informational



Fixed

7. Weaknesses

This section contains the list of discovered weaknesses.

HOLD1-1 | Transaction data handling does not match privacy policy disclosures

Fixed 

Severity:

Low

Description:

The Privacy Policy states that Valigator does not log, store, or transmit wallet addresses, transaction data, or on-chain activity.

- [tooth/docs/PRIVACY_POLICY.md:104](#)

Valigator does not log, store, or transmit your wallet addresses, transaction data, or on-chain activity.

The RPC proxy does not fully match this statement for **sendTransaction** requests. It defines a server-side transactions table that can store transaction-specific fields, including the raw request body, source IP, wallet address, stake account, amount, destination account, validator, and transaction signature.

- [rpc-proxy/src/schema.sql:6](#)

```
CREATE TABLE IF NOT EXISTS transactions (  
  signature TEXT,  
  ip TEXT,  
  raw_body TEXT,  
  wallet_address TEXT,  
  stake_account TEXT,  
  amount_lamports TEXT,  
  operation TEXT,  
  destination_account TEXT,  
  validator TEXT,  
  network TEXT  
);
```

The request handler then decodes the transaction body and writes those fields into the table.

- `rpc-proxy/src/index.ts:3713`

```
`INSERT INTO transactions (signature, method, ip, helius_status, raw_body,  
client_version, error, wallet_address, stake_account, amount_lamports,  
operation, destination_account, validator, network)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`
```

The same transaction metadata is also used to build operational alert messages. These messages can include wallet address, stake account, validator, and transaction signature.

- `rpc-proxy/src/slack-utils.ts:124`

```
if (meta.walletAddress) lines.push(`Wallet:  
\\${sanitize(meta.walletAddress)}\\`);  
pushStakeLines(lines, meta);  
if (meta.validator) lines.push(`Validator:  
\\${sanitize(meta.validator)}\\`);  
  
const sig = sigLine(txSignature, network);  
if (sig) lines.push(sig);
```

Those alert messages are sent through the configured webhook path.

- `rpc-proxy/src/slack-utils.ts:248`

```
await fetch(webhookUrl, {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ text: message }),  
});
```

No application-level cleanup path was identified for pruning old rows from the transactions table. As implemented, the proxy retains and forwards transaction-specific metadata in ways that do not match the narrower data-handling scope described in the Privacy Policy.

Remediation:

- Remove or minimize transaction-specific fields written to the transactions table.
- Avoid storing raw JSON-RPC request bodies unless strictly required.
- Add scheduled pruning for transaction rows older than the intended retention window.
- Redact wallet addresses, stake accounts, transaction signatures, and lamport amounts from operational alert payloads.
- Keep operational alerts limited to non-user-specific status and error categories.

Verification Notes:

It was confirmed that PII has been removed from the D1 transactions table and Slack notifications. It was also verified that no PII fields are used in the aggregated metrics.

HOLD1-2 | WebSocket upgrades rely on a spoofable Origin header for access control

Fixed 

Severity:

Informational

Description:

The RPC proxy handles WebSocket upgrades differently from normal HTTP requests. For WebSocket traffic, the app token and HMAC checks are skipped, leaving the Origin allowlist as the only request-level access check.

- rpc-proxy/src/index.ts

```
if (env.ALLOWED_ORIGINS) {  
  const origin = request.headers.get('Origin');  
  if (origin) {  
    const allowedList = env.ALLOWED_ORIGINS.split(',').map(o => o.trim());  
    if (!allowedList.includes(origin)) {  
      failures.push('origin');  
    }  
  } else if (isWebSocket || !env.HMAC_SECRET) {  
    failures.push('origin');  
  }  
}
```

...

```
if (env.APP_TOKEN && !isWebSocket) {  
  const clientToken = request.headers.get('X-Holdfast-Token') ?? '';  
  if (!timingSafeEqual(clientToken, env.APP_TOKEN)) {  
    failures.push('app-token');  
  }  
}
```

...

```
if (env.HMAC_SECRET && !isWebSocket) {  
  const timestampStr = request.headers.get('X-Holdfast-Timestamp');  
  const clientSigHex = request.headers.get('X-Holdfast-Signature');
```

The proxy then connects to the upstream Helius WebSocket endpoint using the server-side API key. Since non-browser clients can supply their own Origin header, the current check does not provide a complete credential for WebSocket access.

```
const upstreamUrl = `wss://mainnet.helius-rpc.com${search ? `${search}&` :  
'?'}api-key=${env.HELIUS_API_KEY}`;
```

The WebSocket path is limited to subscription methods, but it can still allow unauthenticated use of the proxy's upstream WebSocket access.

```
const ALLOWED_WS_METHODS = new Set([  
  'signatureSubscribe',  
  'signatureUnsubscribe',  
  'slotSubscribe',  
  'slotUnsubscribe',  
]);
```

Remediation:

- Require a WebSocket-specific credential in addition to the Origin header.
- Issue short-lived WebSocket tickets from an authenticated HTTP endpoint.
- Validate the ticket during the WebSocket upgrade before connecting upstream.
- Add per-IP WebSocket connection and subscription limits.

Verification Notes:

It was confirmed that the WebSocket bridge has been completely removed and that all WebSocket upgrade requests are rejected with an HTTP 426 response.

HOLD1-3 | Privacy mode is not consistently applied to transaction history amounts

Fixed ✓

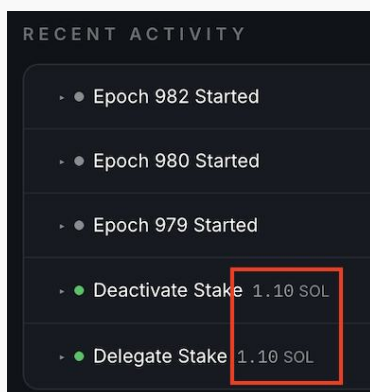
Severity:

Informational

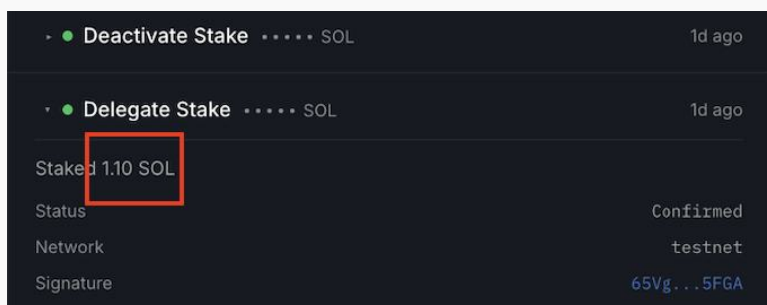
Description:

Transaction history entries can store amount-bearing summaries, such as staking or withdrawal amounts. When privacy mode is enabled, these values are still rendered in some history views.

The Dashboard Recent Activity section does not consume privacyMode and renders transaction amounts and summaries directly.



The Activity Log applies privacy masking to some fields, but expanded rows still render tx.summary directly.



As a result, amount-bearing transaction history text can remain visible in Dashboard Recent Activity and expanded Activity Log entries while privacy mode is enabled. Some masked rows also continue to show the SOL unit label, which should be aligned with the existing masking convention.

Remediation:

- Apply privacy-mode masking in RecentActivitySection.
- Avoid rendering preformatted amount-bearing tx.summary strings while privacy mode is enabled.
- Rebuild history summaries from mask-aware fields where amounts are displayed.
- Align SOL unit rendering with the existing masked amount convention.

Verification Notes:

*It was confirmed that **tx.summary** is rendered only when **!privacyMode** in both the Dashboard and Activity Log.*

HOLD1-4 | Withdraw amount prefill can generate a value rejected by its parser

Fixed 

Severity:

Informational

Description:

The partial withdraw form pre-fills the amount input from the account balance. That value is produced with `formatSol(..., 9)`, which adds thousands separators for large SOL amounts.

- tooth/packages/extension/src/views/WithdrawIntro.tsx

```
function lamportsToSolInput(lamports: bigint): string {
  const formatted = formatSol(lamports, 9);
  if (!formatted.includes('.')) return formatted;
  return formatted.replace(/\.?0+$/, "");
}
```

- tooth/packages/core/src/format/sol.ts

```
function addThousandsSeparator(n: string): string {
  return n.replace(/\B(?=(\d{3})+(?!))/g, ',');
}
```

The same value is later parsed by `parseSolToLamports`, which only accepts digits and a decimal point. A generated value such as 1,000 is therefore rejected as invalid.

- tooth/packages/extension/src/views/withdraw-amount-parser.ts

```
export function parseSolToLamports(input: string): bigint {
  const trimmed = input.trim();
  if (!/^\d+(\.\d+)?|\.\d+$/.test(trimmed)) {
    throw new Error('Invalid number');
  }
}
```

This affects the withdraw amount form when the generated prefill or Max value contains commas. The transaction is not built while validation fails, but the app can reject its own generated input.

Withdraw to wallet

WITHDRAWING

Stake account	So1a...1111	 
Account balance	1,500.0000	SOL
Destination wallet	Gs4L...TkTK	 

WITHDRAW AMOUNT

1,500

SOL

Max

Invalid number - use digits and one decimal point.

Continue

Cancel

Remediation:

- Use a comma-free formatter for editable SOL input values.
- Keep display formatting separate from parser-compatible input formatting.
- Ensure the Max button writes a parser-compatible value.

Verification Notes:

It was confirmed that a comma free formatter is used for editable input fields and that the display format is separated from the parser compatible format.

HOLD1-5 | Split preflight integrity check does not validate the account-creation prelude

Fixed 

Severity:

Informational

Description:

The split flow creates a transaction that includes a **System.createAccount** prelude and a **Stake.Split** instruction. The preflight integrity check validates the stake instruction, but it does not decode or validate the accompanying account-creation fields.

- `tooth/packages/core/src/stake/operations/split.ts`

```
const splitInstruction = StakeProgram.split(  
  {  
    stakePubkey: stakeAccount,  
    authorizedPubkey: stakeAuthority,  
    splitStakePubkey: splitKeypair.publicKey,  
    lamports: safeToNumber(splitLamports),  
  },  
  safeToNumber(RENT_EXEMPT_MINIMUM),  
);
```

- `tooth/packages/extension/src/services/preflight-integrity.ts`

```
case 'split': {  
  if (BigInt(decoded.lamports) !== params.splitLamports) {  
    fail('lamports', 'split lamports mismatch');  
  }  
  return;  
}
```

The **create-delegate** path already performs explicit checks for the corresponding **System.createAccount** prelude. The split path should apply the same consistency checks so the full transaction structure is covered by preflight validation.

- `tooth/packages/extension/src/services/preflight-integrity.ts`

```

if (!pubkeyEq(createParams.fromPubkey, walletPubkey)) {
  fail('source_key', 'System.createAccount fromPubkey != walletPubkey');
}

if (!pubkeyEq(createParams.newAccountPubkey, initParams.stakePubkey)) {
  fail('source_key', 'System.createAccount newAccountPubkey != stake account');
}

if (BigInt(createParams.lamports) !== params.amountLamports) {
  fail('lamports', 'System.createAccount lamports != params.amountLamports');
}

```

Review Transaction

Split 1.00 SOL from FEL5...1sDK. Source retains 2.00 SOL.

SPLIT STAKE	
Source	FEL5...1sDK
Split amount	1.00 SOL
Source retains	2.00 SOL
Estimated fee	0.00 SOL

Verify these details match what your hardware wallet displays before confirming on the device.

Your device may show this as a raw transaction. Verify the program ID is Stake11111...1111 and the amounts match the summary above.

Sign Transaction

Cancel

Remediation:

- Decode the split transaction's **System.createAccount** instruction during preflight validation.
- Validate the funder, new account, lamports, owner, and account space against the expected split transaction.

Verification Notes:

*It was confirmed that validation of the **System.createAccount** prelude has been implemented as part of the split preflight integrity checks.*

hexens x H Holdfast

